МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему
«Розробка інтерфейсу користувача»

Виконав:
студент групи ІМ-55
Стеблевець Олександр Олександрович
Номер в списку – 26

Перевірив:
Рекечинський Д.О.

Київ 2026

Завдання

1. Створити у середовищі розробки проєкт з ім’ям Lab3 (продовження проєкту графічного редактора, розробленого у лабораторній роботі №2).
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Завдання згідно варіанту

1. У звіті повинна бути схема успадкування класів – діаграма класів.
2. Усі методи-обробники повідомлень повинні бути функціями-членами деякого класу.
3. Для вибору типу об’єкта в графічному редакторі Lab3 передбачено вікно Toolbar з кнопками-іконками відповідно до типів об’єктів. Кнопки мають підказки (tooltips) та дублюють функціональність пунктів меню «Об’єкти», яке зникло з головного вікна: вибір інструмента виконується виключно кнопками панелі.
4. Для вибору варіанту використовується Ж = Ж лаб2 + 1, де Ж лаб2 – номер студента в журналі, використаний у лабораторній роботі №2 (Ж лаб2 = 26), тож Ж = 27. Решта вимог повторює пункти 4–7 лабораторної роботи №2 з новим значенням Ж.

Для мене (Ж = 27) буде:

• динамічний масив вказівників для Shape (27 mod 3 = 0) розрахунковим обсягом N = Ж + 100 = 127 об’єктів;

- «гумовий» слід (27 mod 4 = 3) – пунктирна лінія чорного кольору;
- прямокутник: ввід від центру до одного з кутів (27 mod 2 = 1), чорний контур з кольоровим заповненням (27 mod 5 = 2), колір заповнення – рожевий (27 mod 6 = 3);
- еліпс: ввід по двом протилежним кутам охоплюючого прямокутника (27 mod 2 = 1), чорний контур без заповнення (27 mod 5 = 2);
- позначка поточного типу об’єкта: у заголовку вікна (27 mod 2 = 1), дублюється підсвічуванням відповідної кнопки Toolbar.

Лістинги програми

```dart
import 'package:flutter/material.dart';
import 'package:window_manager/window_manager.dart';

import 'editor/editor_page.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await windowManager.ensureInitialized();
  runApp(const PaintApp());
}

class PaintApp extends StatelessWidget {
  const PaintApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Paint',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const EditorPage(title: 'Paint'),
    );
  }
}
```

Лістинг 1 – main.dart

```dart
import 'package:flutter/material.dart';
import 'package:window_manager/window_manager.dart';

import 'editor_menu.dart';
import 'editor_toolbar.dart';
import 'paint_canvas.dart';
import 'shape/shape_type.dart';

class EditorPage extends StatefulWidget {
  const EditorPage({super.key, required this.title});

  final String title;

  @override
  State<EditorPage> createState() => _EditorPageState();
}

class _EditorPageState extends State<EditorPage> {
  ShapeType? selectedTool;
```

```
  @override
  void initState() {
    super.initState();
    _updateTitle();
  }

  void _selectTool(ShapeType? tool) {
    setState(() => selectedTool = tool);
    _updateTitle();
  }

  void _updateTitle() {
    final tool = selectedTool;
    windowManager.setTitle(tool == null ? 'Paint' : 'Paint (${tool.label})');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // appBar: AppBar(title: Text(widget.title)),
      body: Column(
        children: [
          Align(
            alignment: AlignmentDirectional.centerStart,
            child: EditorMenu(selectedTool: selectedTool, onToolSelected: _selectTool),
          ),
          EditorToolbar(selectedTool: selectedTool, onToolSelected: _selectTool),
          Expanded(child: PaintCanvas(selectedTool: selectedTool)),
        ],
      ),
    );
  }
}
```

Лістинг 2 – editor_page.dart

```
import 'package:flutter/material.dart';
import 'package:paint/editor/shape/shape_type.dart';

class EditorMenu extends StatelessWidget {
  const EditorMenu({
    super.key,
    required this.selectedTool,
    required this.onToolSelected,
  });

  final ShapeType? selectedTool;
  final ValueChanged<ShapeType?> onToolSelected;

  @override
  Widget build(BuildContext context) {
    return MenuBar(
      children: [
        SubmenuButton(
          menuChildren: const [],
          child: const Text('Файл'),
        ),
        SubmenuButton(
          menuChildren: [
            for (final ShapeType tool in ShapeType.values)
              CheckboxMenuButton(
                value: selectedTool == tool,
                onChanged: (bool? selected) => onToolSelected(selected == true ? tool : null),
                child: Text(tool.label),
              ),
          ],
          child: const Text('Об\'єкти'),
        ),
        SubmenuButton(
          menuChildren: const [],
          child: const Text('Довідка'),
        ),
      ],
    );
```

```
    }
  }
```

Лістинг 3 – editor_menu.dart

```
import 'package:flutter/material.dart';

import 'shape/shape_type.dart';

class EditorToolbar extends StatelessWidget {
  const EditorToolbar({
    super.key,
    required this.selectedTool,
    required this.onToolSelected,
  });

  final ShapeType? selectedTool;
  final ValueChanged<ShapeType?> onToolSelected;

  static IconData _iconFor(ShapeType type) => switch (type) {
    ShapeType.dot => Icons.trip_origin,
    ShapeType.line => Icons.horizontal_rule,
    ShapeType.rectangle => Icons.crop_square,
    ShapeType.ellipse => Icons.panorama_fish_eye,
  };

  Widget _buildButton(ColorScheme colorScheme, ShapeType tool) {
    final selected = selectedTool == tool;
    return IconButton(
      icon: Icon(_iconFor(tool)),
      tooltip: tool.label,
      isSelected: selected,
      onPressed: () => onToolSelected(selected ? null : tool),
      style: ButtonStyle(
        backgroundColor: WidgetStatePropertyAll<Color?>(
          selected ? colorScheme.secondaryContainer : null,
        ),
        foregroundColor: WidgetStatePropertyAll<Color?>(
          selected ? colorScheme.onSecondaryContainer : null,
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return Material(
      color: colorScheme.surfaceContainer,
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 2),
        child: Row(
          children: [
            for (final ShapeType tool in ShapeType.values) _buildButton(colorScheme, tool),
          ],
        ),
      ),
    );
  }
}
```

Лістинг 4 – editor_toolbar.dart

```
import 'package:flutter/material.dart';

import 'canvas_shapes.dart';
import 'shape/shape.dart';
import 'shape/shape_type.dart';

class PaintCanvas extends StatefulWidget {
  const PaintCanvas({super.key, required this.selectedTool});

  final ShapeType? selectedTool;

  @override
  State<PaintCanvas> createState() => _PaintCanvasState();
```

```
}

class _PaintCanvasState extends State<PaintCanvas> {
  final CanvasShapes _storage = CanvasShapes();

  Offset? _dragStart;
  Offset? _dragCurrent;

  Shape _buildShape(Offset start, Offset end) => Shape.of(widget.selectedTool!, start: start, end:
end);

  void _handleTapDown(TapDownDetails details) {
    if (widget.selectedTool != ShapeType.dot) {
      return;
    }
    final position = details.localPosition;
    _storage.add(_buildShape(position, position));
  }

  void _handlePanStart(DragStartDetails details) {
    if (widget.selectedTool == ShapeType.dot) {
      return;
    }
    setState(() {
      _dragStart = details.localPosition;
      _dragCurrent = details.localPosition;
    });
  }

  void _handlePanUpdate(DragUpdateDetails details) {
    if (widget.selectedTool == ShapeType.dot) {
      return;
    }
    setState(() {
      _dragCurrent = details.localPosition;
    });
  }

  void _handlePanEnd(DragEndDetails details) {
    if (widget.selectedTool == ShapeType.dot) {
      return;
    }
    final start = _dragStart;
    final current = _dragCurrent;
    setState(() {
      _dragStart = null;
      _dragCurrent = null;
    });
    if (start == null || current == null) {
      return;
    }
    _storage.add(_buildShape(start, current));
  }

  @override
  Widget build(BuildContext context) {
    final tool = widget.selectedTool;
    return GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTapDown: _handleTapDown,
      onPanStart: _handlePanStart,
      onPanUpdate: _handlePanUpdate,
      onPanEnd: _handlePanEnd,
      child: ColoredBox(
        color: Colors.white,
        child: CustomPaint(
          painter: _CanvasPainter(
            storage: _storage,
            tool: widget.selectedTool,
            dragStart: _dragStart,
            dragCurrent: _dragCurrent,
          ),
          child: const SizedBox.expand(),
        ),
      ),
    );
  }
```

```
}

class _CanvasPainter extends CustomPainter {
  const _CanvasPainter({
    required CanvasShapes storage,
    required this.tool,
    required this.dragStart,
    required this.dragCurrent,
  }) : _storage = storage,
       super(repaint: storage);

  final CanvasShapes _storage;
  final ShapeType? tool;
  final Offset? dragStart;
  final Offset? dragCurrent;

  @override
  void paint(Canvas canvas, Size size) {
    canvas.clipRect(Offset.zero & size);
    for (final shape in _storage.shapes) {
      shape.paint(canvas);
    }
    final start = dragStart;
    final current = dragCurrent;
    if (start == null || current == null || tool == null) {
      return;
    }
    Shape.of(tool!, start: start, end: current).paintPreview(canvas);
  }

  @override
  bool shouldRepaint(covariant _CanvasPainter oldDelegate) =>
      oldDelegate.dragStart != dragStart || oldDelegate.dragCurrent != dragCurrent;
}
```

Лістинг 5 – paint_canvas.dart

```
import 'package:flutter/foundation.dart';

import 'shape/shape.dart';

class CanvasShapes extends ChangeNotifier {
  final List<Shape> shapes = <Shape>[];

  int get count => shapes.length;

  void add(Shape shape) {
    shapes.add(shape);
    notifyListeners();
  }
}
```

Лістинг 6 – canvas_shapes.dart

```
enum ShapeType { dot, line, rectangle, ellipse }

extension ShapeTypeLabel on ShapeType {
  String get label => switch (this) {
    ShapeType.dot => 'Крапка',
    ShapeType.line => 'Лінія',
    ShapeType.rectangle => 'Прямокутник',
    ShapeType.ellipse => 'Еліпс',
  };
}
```

Лістинг 7 – shape_type.dart

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import 'dot_shape.dart';
import 'ellipse_shape.dart';
import 'line_shape.dart';
import 'rectangle_shape.dart';
```

```dart
import 'shape_type.dart';

abstract class Shape {
  const Shape({required this.start, required this.end});

  static const double _dash = 6;
  static const double _gap = 4;

  final Offset start;
  final Offset end;

  factory Shape.of(ShapeType type, {required Offset start, Offset? end}) =>
      switch (type) {
        ShapeType.dot => DotShape(start: start),
        ShapeType.line => LineShape(start: start, end: end ?? start),
        ShapeType.rectangle => RectangleShape(start: start, end: end ?? start),
        ShapeType.ellipse => EllipseShape(start: start, end: end ?? start),
      };

  void drawDashed(Canvas canvas, Paint paint, Path path) {
    for (final metric in path.computeMetrics()) {
      double distance = 0;
      while (distance < metric.length) {
        final next = math.min(distance + _dash, metric.length);
        canvas.drawPath(metric.extractPath(distance, next), paint);
        distance = next + _gap;
      }
    }
  }

  void paint(Canvas canvas);

  void paintPreview(Canvas canvas);
}
```

Лістинг 8 – shape.dart

```dart
import 'package:flutter/material.dart';

import 'shape.dart';

class DotShape extends Shape {
  const DotShape({required super.start}) : super(end: start);

  @override
  void paint(Canvas canvas) {
    canvas.drawCircle(start, 3, Paint()..color = Colors.black);
  }

  @override
  void paintPreview(Canvas canvas) {}
}
```

Лістинг 9 – dot_shape.dart

```dart
import 'package:flutter/material.dart';

import 'shape.dart';

class LineShape extends Shape {
  const LineShape({required super.start, required super.end});

  @override
  void paint(Canvas canvas) {
    canvas.drawLine(start, end, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(
      canvas,
      _stroke(Colors.black),
      Path()
        ..moveTo(start.dx, start.dy)
        ..lineTo(end.dx, end.dy),
    );
```

```
    }

    static Paint _stroke(Color color) =>
        Paint()
          ..color = color
          ..style = PaintingStyle.stroke
          ..strokeWidth = 2;
}
```

Лістинг 10 – line_shape.dart

```
import 'package:flutter/material.dart';

import 'shape.dart';

class RectangleShape extends Shape {
  const RectangleShape({required super.start, required super.end});

  Rect get _rect => Rect.fromCenter(
    center: start,
    width: (end.dx - start.dx).abs() * 2,
    height: (end.dy - start.dy).abs() * 2,
  );

  Paint get _fill => Paint()..color = Colors.pink;

  Paint _stroke(Color color) =>
      Paint()
        ..color = color
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2;

  @override
  void paint(Canvas canvas) {
    canvas.drawRect(_rect, _fill);
    canvas.drawRect(_rect, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(canvas, _stroke(Colors.black), Path()..addRect(_rect));
  }
}
```

Лістинг 11 – rectangle_shape.dart

```
import 'package:flutter/material.dart';

import 'shape.dart';

class EllipseShape extends Shape {
  const EllipseShape({required super.start, required super.end});

  Rect get _rect => Rect.fromPoints(start, end);

  Paint _stroke(Color color) =>
      Paint()
        ..color = color
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2;

  @override
  void paint(Canvas canvas) {
    canvas.drawOval(_rect, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(canvas, _stroke(Colors.black), Path()..addOval(_rect));
  }
}
```

Лістинг 12 – ellipse_shape.dart

Діаграма

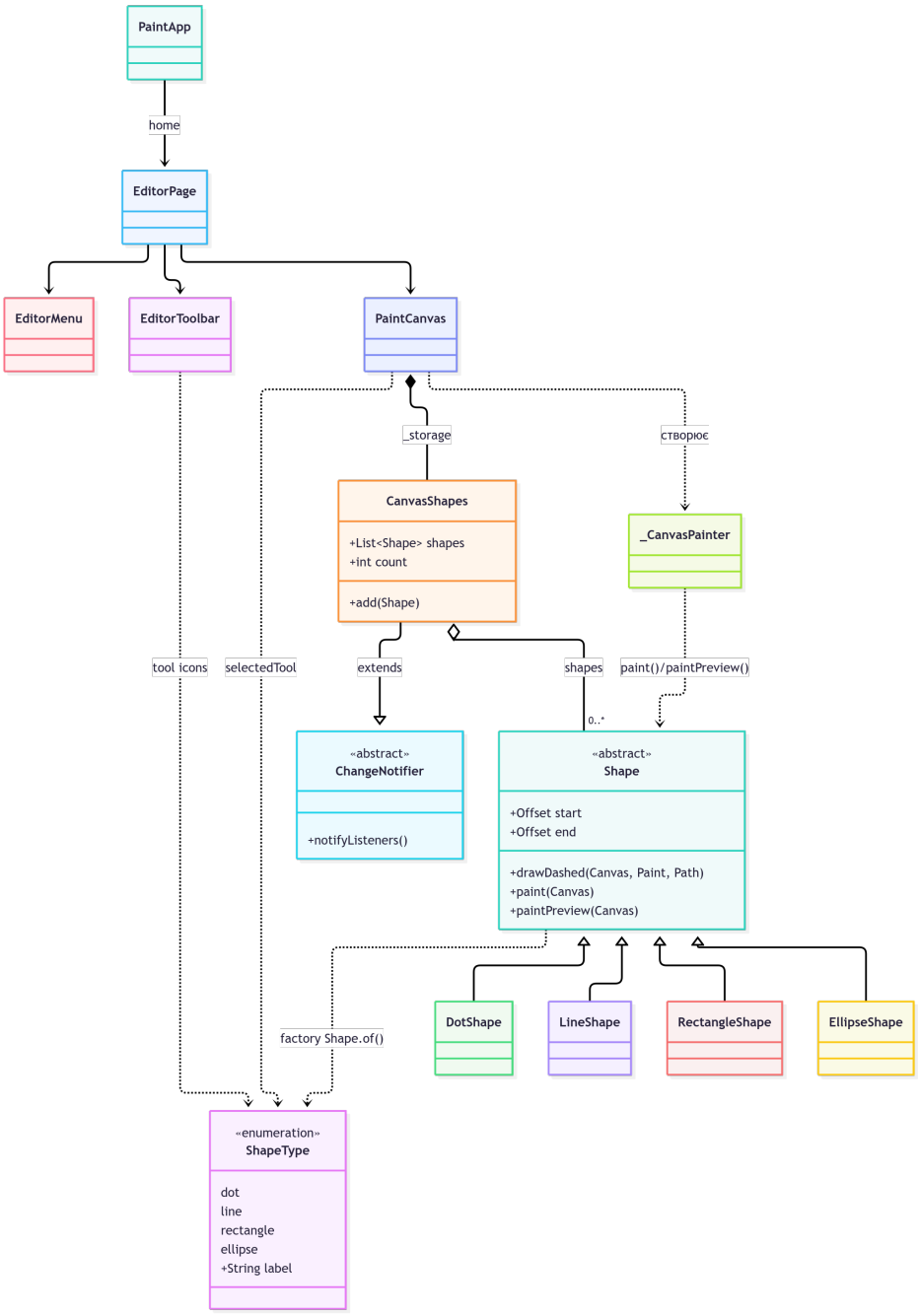


Рисунок 1 – Діаграма класів графічного редактора

Скріншоти

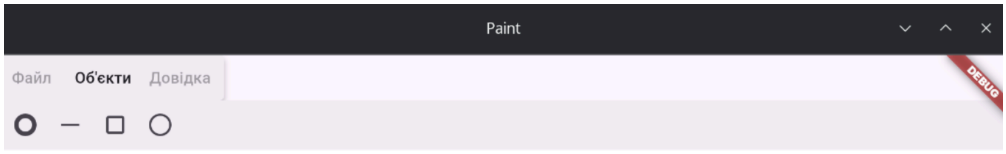


Рисунок 2 – Вигляд програми після запуску

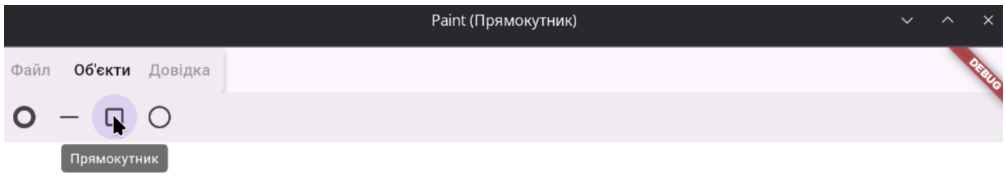


Рисунок 3 – Вибір інструмента «Прямокутник»: підсвічена кнопка, підказка (tooltip) та позначка поточного типу об’єкта у заголовку вікна

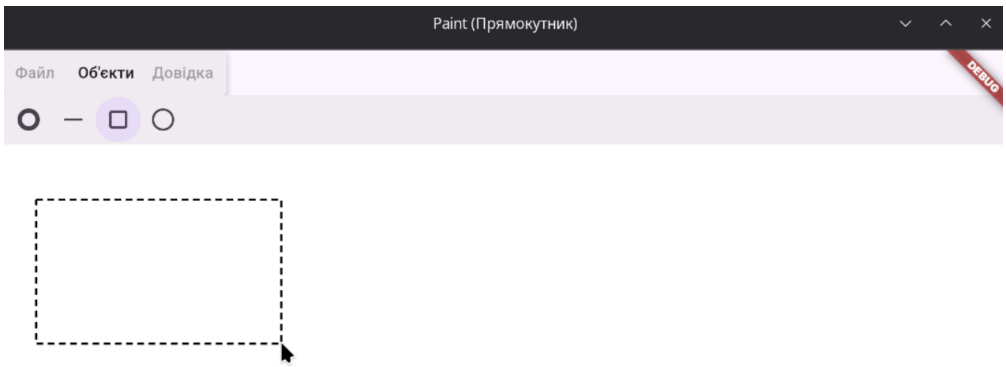


Рисунок 4 – Ввід прямокутника від центру до кута: пунктирний «гумовий» слід під час перетягування; частина фігури, що вийшла за межі полотна, обрізається

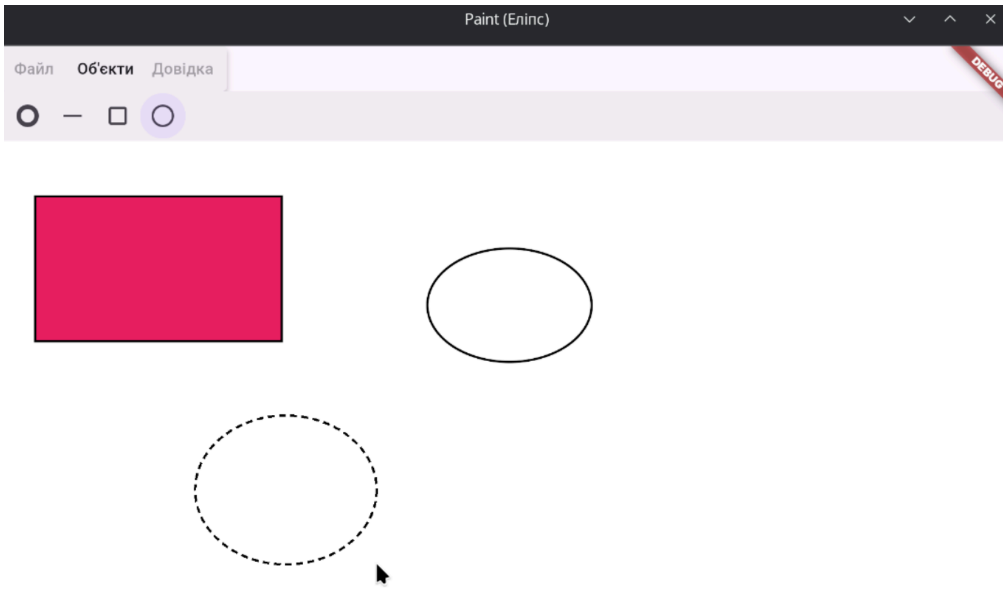


Рисунок 5 – Результат роботи: рожеві прямокутники з чорним контуром, еліпс без заповнення, ввід еліпса по протилежних кутах охоплюючого прямокутника

Висновки

Отримав досвід роботи з елементом toolbar в Flutter. Перемістив частину меню (субменю Об’єкти) на місце тулбару. Поміняв стилі роботи елементів редактора (прямокутник, лінія, крапка, еліпс). Зробив список елементів динамічним.